

Weak Hash Function (weakhash)

To help a friend who is working on the internals of the Java Virtual Machine, Luca is working on a new *hash function*. Hash functions are meant to take as an input a large value, process it according to internal rules, and finally produce a smaller value.

Since the number of possible different output values is limited and smaller than the number of possible input values, by using a hash function one accepts the risk of *collisions*. A collision happens when two distinct input values, when hashed, give the same output value.

Luca's algorithm is rather simple, as the use case admits the possibility of collisions, provided that they don't grow too much. The input number is a sequence of N decimal digits from 1 to 9: $d_1, d_2, \dots, d_N$. The hash is computed by multiplying each digit with the next one and reducing the result using the modulo $M = 10^7 + 19 = 10\,000\,019$, until reaching the last digit. Formally:

$$\text{hash} = (((\dots(((d_1 \cdot d_2) \bmod M) \cdot d_3) \bmod M) \dots) \cdot d_N) \bmod M$$

While performing some experiments, a specific output value H seems to be occurring way more often than all other values. Luca is now worried that for some reason there might be a lot of different inputs that produce exactly H when hashed. How many such inputs are there?

📎 Among the attachments of this task you may find a template file `weakhash.*` with a sample incomplete implementation.

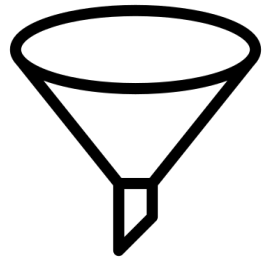


Figure 1: Hash functions are often represented with a funnel (large input, small output).

Input

The first line contains two integers, N and H .

Output

You need to write a single line with an integer: the number of different values that produce H when hashed with Luca's algorithm. Since this number may be large, report it **modulo 10 000 019**.

📎 The *modulo* operation ($a \bmod m$) can be written in C++ as `(a % m)`. To avoid the *integer overflow* error, remember to reduce all partial results through the modulus, and not just the final result!
 Notice that if $x < 10^7 + 19$, then $200x$ fits into a C++ `int` but for holding x^2 you need a `long long`.

Constraints

- $1 \leq N \leq 50$.
- $1 \leq H < 10\,000\,019$.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

- **Subtask 1** (0 points) Examples.
 
- **Subtask 2** (7 points) $N = 1$.
 
- **Subtask 3** (23 points) $N \leq 4$.
 
- **Subtask 4** (8 points) $N \leq 7$.
 
- **Subtask 5** (24 points) $N \leq 15$.
 
- **Subtask 6** (29 points) $N \leq 30$.
 
- **Subtask 7** (9 points) No additional limitations.
 

Examples

input	output
1 4	1
2 8	4

Explanation

In the **first sample case** there is just one number (4) with $N = 1$ digits and whose hash is $H = 4$.
In the **second sample case**, there are four numbers (18, 24, 42, and 81) with $N = 2$ digits and whose hash is $H = 8$.